

Phase 1: Planning & Architecture

Generative AI Pipeline Design on Databricks

1. Translating Business Use Cases to Pipeline Inputs & Outputs

Core Concept

Before writing code, you must translate business requirements into specific architectural inputs and outputs. A GenAI app generally takes user input and uses foundation models to create new outputs, automate tasks, or engage in intelligent interactions *[cite: <https://docs.databricks.com/aws/en/agents/concepts/>]*. The architectural output format depends on the deployment pattern: a chat app (Databricks Apps), an API endpoint (Model Serving), or a SQL function for analysts (AI Functions)

[cite: <https://docs.databricks.com/aws/en/agents/concepts/>].

How It Works on Databricks

When bridging general intelligence to specific business processes (e.g., converting unstructured B2B bakery wholesale inquiries into database records), relying on arbitrary text output is fragile. Databricks supports **Structured Outputs** via Foundation Model APIs. This enforces the model to return JSON objects that adhere strictly to a defined JSON schema

[cite: <https://docs.databricks.com/aws/en/machine-learning/model-serving/structured-outputs/>]. This is heavily recommended when extracting data from large amounts of documents to ensure downstream data pipelines do not break *[cite: <https://docs.databricks.com/aws/en/machine-learning/model-serving/structured-outputs/>]*.

When to Use vs. Alternatives

- **Use Structured Outputs (JSON Schema):** When the output must feed directly into programmatic marketing tools, Make.com webhooks, or relational databases where schema mismatches cause failures.
- **Alternative (AI Functions in SQL):** If the goal is purely analytical (e.g., scoring lead sentiment directly in a Lakehouse table), use built-in SQL AI Functions instead of a standalone API endpoint

[cite: <https://docs.databricks.com/aws/en/agents/concepts/>].

Common Mistakes

A frequent error is failing to isolate inputs/outputs logically or legally. Remember that inputs and outputs deployed via Databricks Model Serving are isolated from other customers and stored for up to 30 days for security monitoring, but depending on the partner model provider (like OpenAI), data retention policies vary *[cite: <https://docs.databricks.com/aws/en/machine-learning/model-serving/>]*. Designing pipelines with sensitive customer financial data without verifying model provider data retention is a critical compliance failure.

Code Example & Function Explanation

The following example uses the Databricks Chat API leveraging the `response_format` parameter to guarantee a structured output for a lead generation pipeline.

```
from openai import OpenAI
import os

# Initialize Databricks Foundation Model client
client = OpenAI(
    api_key=os.environ.get("DATABRICKS_TOKEN"),
    base_url="https://serving-endpoints"
)

# Define the precise schema required for downstream webhooks
schema = {
    "type": "object",
    "properties": {
        "company_name": {"type": "string"},
        "interest_level": {"type": "string", "enum": ["High", "Medium", "Low"]},
        "requested_products": {"type": "array", "items": {"type": "string"}}
    },
    "required": ["company_name", "interest_level", "requested_products"]
}

# Explain function used: chat.completions.create
# This function sends the prompt to the served model. By passing the schema
# to 'response_format', we force the LLM to output valid JSON matching our pipeline
# needs.
response = client.chat.completions.create(
    model="databricks-meta-llama-3-70b-instruct",
    messages=[{"role": "user", "content": "Extract lead info: The H bakery received an inquiry from Solaria Properties for 50 tin-box cakes."}],
    response_format={"type": "json_object", "schema": schema}
)
print(response.choices[0].message.content)
```

2. Selecting Model Tasks for Business Requirements

Core Concept

Once inputs and outputs are defined, you must select the appropriate level of agent autonomy to perform the task. Design patterns form a continuum of complexity: from deterministic chains, through single-agent systems capable of dynamic decisions, up to multi-agent architectures that coordinate specialized agents *[cite: <https://docs.databricks.com/aws/en/agents/agent-system-design-patterns>]*.

How It Works on Databricks

You select tasks by equipping the GenAI model with specific **Tools**. Databricks categorizes tools into three main types [cite: <https://docs.databricks.com/aws/en/agents/concepts/>]:

- **Data Retrieval:** Semantic retrieval (querying a vector index) or Structured retrieval (running SQL queries).
- **State Modification:** API calls (e.g., triggering a CRM or email webhook).
- **Logic/Execution:** Code execution (running Python code in a sandbox).

For example, a retail banking loan assistant agent follows a reasoning flow: it *plans* (check client history), uses a *structured retrieval tool* (SQL query on loan database), *reasons* on the output, and executes an *action* (calls an API to generate a document)

[cite: <https://docs.databricks.com/aws/en/agents/agent-system-design-patterns/>].

When to Use vs. Alternatives

- **Use Deterministic Chains:** For predictable workflows like summarization or basic translation where the steps are fixed.
- **Use Single/Multi-Agent Systems:** When the business process requires "Human-in-the-Loop" fallback, dynamic data gathering, or complex constrained portfolio optimization where the LLM must write and test code to find an answer

[cite: <https://docs.databricks.com/aws/en/agents/agent-system-design-patterns/>].

Documentation Verification Note: The Databricks documentation clearly outlines the continuum from single to multi-agent architectures and the use of tools. However, specific point-and-click UI documentation for configuring an out-of-the-box "Information Extraction Agent Brick" cannot be verified in the core documentation; extraction is generally treated as a pipeline pattern using Structured Outputs rather than a distinct agent UI feature.

Common Mistakes

Applying a multi-agent orchestration architecture when a simple deterministic chain or PySpark pipeline would suffice. Unnecessary agentic autonomy increases latency, cost, and the risk of hallucinated tool calls. Furthermore, failing to implement proper evaluation for complex open-ended agent behavior is a common pitfall; Databricks notes that bridging general and data intelligence requires specialized AI evaluation tools alongside development [cite: <https://docs.databricks.com/aws/en/agents/concepts/>].

Code Example & Function Explanation

This conceptual snippet demonstrates defining a structured retrieval tool for an agent using Databricks' ecosystem concepts.

```

from databricks.sdk import WorkspaceClient

# Explain function used: defining a Tool function for the Agent.
# In a LangChain/Databricks agent architecture, functions are bound to the LLM.
# The LLM reasons about the user query and decides if it needs to execute this SQL
tool.
def structured_retrieval_tool(client_id: str) -> str:
    """Tool to retrieve a client's risk profile from the retail banking Databricks
    SQL warehouse."""
    w = WorkspaceClient()
    # Execute SQL via Databricks SDK Statement Execution API
    # This represents a "Structured Retrieval" task selected for the agent.
    res = w.statement_execution.execute_statement(
        warehouse_id="",
        statement=f"SELECT risk_score FROM silver_retail_clients WHERE id =
'{client_id}'"
    )
    return str(res.result.data_array)

# The agent would then be instantiated with this tool, allowing it to bridge
# its general reasoning with specific enterprise data intelligence.

```